

Detailed and Updated Summary:
*An improved frontier-based robot exploration strategy combined
with deep reinforcement learning*
Robotics and Autonomous Systems 181 (2024) 104783

NECKER

Based on the original PDF + some clarifications

December 9, 2025

Abstract

This document presents a detailed analysis of a new autonomous exploration strategy for mobile robots. The proposed approach combines the traditional frontier-based method with Deep Reinforcement Learning (DRL) for optimal target point selection. The method introduces an innovative neural network (LFPN) that processes both a resized global map and local maps at original resolution, mitigating the loss of information caused by map downsampling. Experimental results, both in simulation and on real robots, demonstrate that this method significantly reduces the travel distance required to complete the exploration compared to existing approaches.

Contents

1	Introduction	2
2	Overview of Methodologies	2
3	Proposed Methodology	3
3.1	System Architecture	3
3.2	Frontier Detection and Decision-Making Process	3
3.2.1	Identification and Filtering of Candidates	3
3.2.2	Selection Criterion: Q-Value Maximization	4
3.3	Neural Network Structure (LFPN)	5
3.3.1	1. Global Network	5
3.3.2	2. Local Network	6
3.3.3	3. Output Network (Evaluation)	7
3.4	Detailed Decision-Making Process	7
4	Reinforcement Learning Environment	8
4.1	Network Training via Double Deep Q-Learning (DDQN)	8
4.1.1	Dual Network Configuration	8
4.1.2	Loss Function and Optimization	8
4.1.3	Soft Target Update	8
4.1.4	Data Management and Training Loop	9
5	Experiments and Results	10
5.1	Experimental Setup	10
5.2	Comparative Results	10
5.3	Physical Experiments	10

1 Introduction

Autonomous exploration requires a robot to move through an unknown environment while building a usable map for subsequent navigation. The primary goal is to minimize the distance traveled while constructing a complete map. Traditionally, *frontier-based* methods guide the robot toward the boundaries between free space and unknown space (frontiers). However, simple methods such as choosing the nearest frontier via Depth-First-Search (DFS) can lead to inefficient paths or collisions if obstacles are close to the target.

With the advent of Deep Learning (DL) and Deep Reinforcement Learning (DRL), solutions have been proposed to handle the high dimensionality of robotic decision-making problems. However, existing DRL methods often require resizing the map to fit the input of the neural network, causing a loss of resolution and critical details, especially in large environments.

The method proposed in this study introduces an improved strategy that uses a neural network to extract both global and local features, allowing the robot to better "understand" the environment and complete the exploration with a lower distance cost.

2 Overview of Methodologies

Before deep learning, the problem was treated as a partially observable Markov decision process (POMDP).

- **Traditional Methods:** Yamauchi (1997) introduced frontier-based exploration. Gonzalez-Baños (2002) improved safety by selecting targets in safe free space. Other methods, such as the one by Bourgault et al., rely on information theory and the maximization of information gain.
- **DRL-Based Methods:** Niroui et al. proposed using DRL with recurrent neural networks (RNNs), but their method requires resizing the global map, losing details. Chen et al. used fixed local maps around the robot, thereby losing the global context of the environment.

3 Proposed Methodology

The core of the proposed strategy is the integration of a traditional SLAM system with a decision-maker based on Deep Reinforcement Learning (DRL). To overcome the limitations of existing architectures, which often suffer from information loss during map resizing or lack global awareness, the **LFPN** (Local Feature Pyramid Network) is introduced. This hybrid architecture simultaneously processes a compressed global view and high-resolution local details, allowing the robot to evaluate exploration targets with a comprehensive contextual understanding.

3.1 System Architecture

The overall architecture is modular and comprises three interconnected components:

1. **SLAM Module (Mapping and Localization):** The *Cartographer* algorithm is employed to process sensor data (LiDAR) in real time. The output is a 2D occupancy grid map, where each cell possesses a discrete state: free space, occupied space (obstacle), or unknown space. This module constantly provides the robot's pose and the updated map m_c .
2. **Navigation Module:** Once a target is selected by the DRL strategy (detailed in subsequent sections), navigation occurs on two levels:
 - **Global Planning:** The A^* algorithm computes the optimal collision-free path from the current position to the target on the static map.
 - **Local Planning:** The *Timed Elastic Band* (TEB) planner optimizes the local trajectory in real time, considering the robot's kinematics and dynamic obstacles, sending velocity commands to the motor controller.
3. **Exploration Strategy (DRL):** This module acts as the decision-making brain. At each step, it analyzes the partially built map, identifies candidate frontier points, and uses the LFPN network to assign a utility score to each, determining the next destination.

3.2 Frontier Detection and Decision-Making Process

The process that drives the robot to move is not based on a simple geometric scan, but on a structured pipeline that transforms raw data into strategic decisions aimed at minimizing the exploration cost.

3.2.1 Identification and Filtering of Candidates

The first step consists of identifying where the robot *could* go. This process takes place on the 2D occupancy grid built by the SLAM module.

- **Definition of Frontier:** A frontier is defined as the boundary separating free space from unknown space on the map.
- **Component Analysis:** The system analyzes the map to locate the boundaries between free and unknown space. Subsequently, it applies connected component analysis to determine the center of each boundary, which becomes a potential target point.
- **Noise Reduction:** To prevent the robot from attempting to explore false spaces generated by sensor noise, all frontiers with a boundary length of less than 0.5 m are discarded.
- **Safe Space:** The robot is not sent exactly to the boundary of the frontier, as it might be too close to obstacles. The free space is eroded to obtain a subset defined as "Safe Space". The final candidate point (action) is chosen by selecting the position within the safe space closest to the center of the frontier boundary.

3.2.2 Selection Criterion: Q-Value Maximization

Once the set of candidates $c = [p_1, \dots, p_N]$ is obtained, the system must choose the best one. Unlike traditional methods that often choose the closest point (Nearest Frontier), the LFPN method chooses based on the predicted overall efficiency.

The Objective Function

The mathematical goal of exploration is to minimize the total distance traveled to complete the map. The problem is modeled as a Markov Decision Process (MDP) where the reward r received at each step is equal to the negative of the distance traveled l :

$$r_i = -l_i \quad (1)$$

Consequently, maximizing the cumulative reward is equivalent to minimizing the total travel distance.

Q-Value Interpretation and Learning Stability

To fully understand the system's operation, it is necessary to clarify the distinction between the immediate physical reward and the network's strategic estimation, as well as the mechanism that ensures learning stability.

Immediate Reward vs. Q-Value: The Network's Foresight

Confusion often arises between the received reward and the network's output.

- **Immediate Reward (r):** This is a real physical value. If the robot travels 5 m to reach a frontier, the reward is $r = -5$. This value only measures the cost of the current action.
- **Q-Value (Output):** This is a long-term prediction. The Q value represents the sum of the immediate reward plus all estimated future rewards until the map is complete:

$$Q \approx r_{\text{now}} + (r_{\text{future}_1} + r_{\text{future}_2} + \dots)$$

Practical Example: Suppose the robot has two options:

1. **Frontier A (Close, 5 m):** It is a dead end. Going there costs little immediately (-5), but it will later force the robot to turn back and take a long detour (future cost -95). The total Q-value will be -100 .
2. **Frontier B (Far, 20 m):** It is the entrance to a central corridor. Going there costs a lot immediately (-20), but it strategically positions the robot to finish the job quickly (future cost -60). The total Q-value will be -80 .

The LFPN system will choose **Frontier B** because $-80 > -100$ (since these are negative numbers, -80 is better), demonstrating that the network accepts a higher immediate cost to achieve a total savings in the long run.

The network learns this estimation by combining the big-picture view of the **Global Network** (which helps plan the order of rooms/areas) with the details of the **Local Network** (which evaluates immediate accessibility). (addressed in the next section)

Greedy Selection

At the end of the evaluation, the system adopts a *greedy* policy. The candidate p_t possessing the maximum Q value among all available options is selected as the next target p_g :

$$p_g = p_t \quad \text{where} \quad t = \arg \max(q_t) \quad (2)$$

This point is then sent to the global planner A^* for navigation.

3.3 Neural Network Structure (LFPN)

The main challenge in using neural networks for exploration is the variability of map dimensions. Resizing the entire map to a fixed dimension drastically reduces resolution, erasing crucial details for safety (small obstacles) and exploration (narrow passages). The LFPN solves this problem by structuring itself into three distinct subnetworks:

3.3.1 1. Global Network

This network aims to extract the overall context of the environment and the topology of unexplored areas.

- **Input Pre-processing:** The current partial map is not inserted raw. The system crops the "effective region" (the rectangle containing the already explored space) and scales it to fit the required size (300×300), enlarging it if it is smaller and shrinking it if it is larger. Finally, it applies padding with "unknown space" values if there are still unexplored areas within the rectangle.
- **Tensor Representation:** The input is a 5-channel tensor ($5 \times 300 \times 300$), where each channel is a binary map (values 0 or 1) representing:
 1. Unknown space.
 2. Free space.
 3. Occupied space (obstacles).
 4. Positions of all detected frontiers.
 5. Current position of the robot.
- **Internal Architecture:** The network utilizes convolutional layers followed by ReLU activations. To mitigate the "limited receptive field" problem typical of standard CNNs (which struggle to connect spatially distant information), a **Self-Attention Module** (inspired by non-local blocks) is inserted. This module computes relationships between distant pixels, allowing the network to understand the global structure.
- **Output:** A global feature vector of dimension 1×200 .

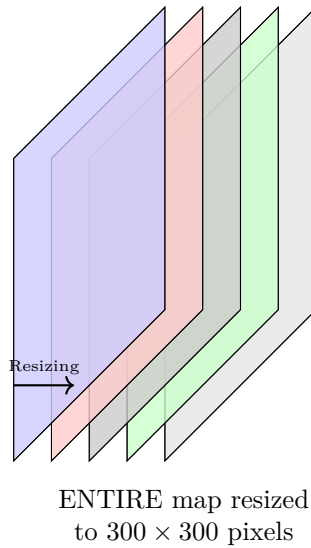
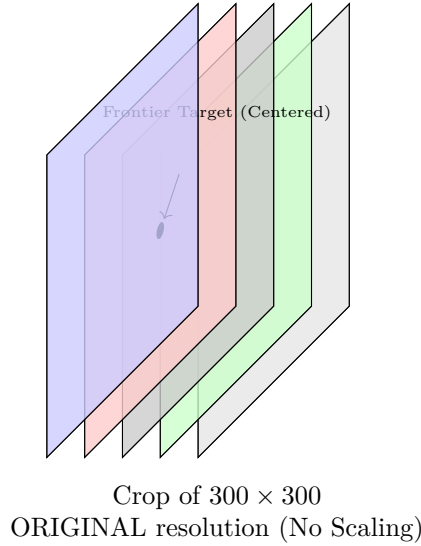


Figure 1: Representation of the Global Input Tensor

3.3.2 2. Local Network

This network analyzes fine details around a specific candidate, ensuring that the robot considers the local geometry (e.g., proximity to obstacles or dead ends).

- **Original Resolution Input:** For each candidate frontier, a map portion (crop) of 300×300 pixels centered exactly on the frontier itself is extracted. Crucially, **no scaling (resizing) is applied**, preserving the original resolution of the occupancy grid.
- **Edge Management:** If the crop exceeds the boundaries of the known map, the external area is padded with the "unknown space" value.
- **Tensor Structure:** Identical to the global network (5 channels). However, in the "Robot Position" channel, a 1 (robot's position) will be present in the grid only if the robot is physically located within the local crop; otherwise, it is an all-zero matrix. In the frontier channel, the center of the crop is always marked as 1 (being the frontier under examination).
- **Weight Sharing:** Although the Local Network shares the exact same structural architecture as the Global Network (including the attention blocks), it possesses distinct parameters (weights), trained specifically to recognize local features.
- **Output:** A local feature vector of dimension 1×200 for the specific frontier analyzed.



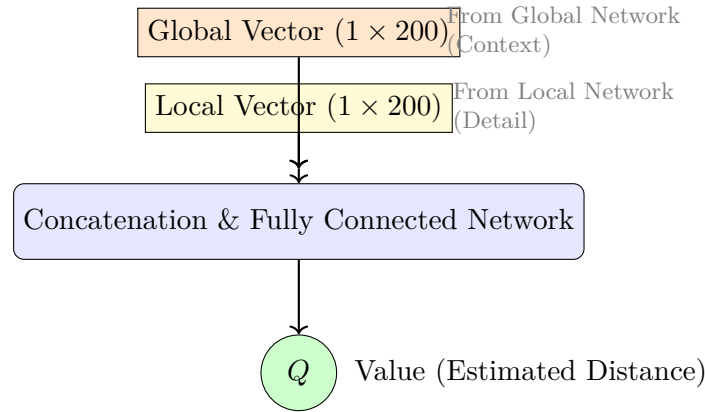
3.3.3 3. Output Network (Evaluation)

This network fuses global and local information to produce the Q-value (expected value of the reward).

- **Input:** It receives as input the concatenation of the global vector (1×200) and the local vector (1×200), resulting in an input vector of 1×400 .
- **Fully Connected Architecture:** The vector passes through a series of linear (densely connected) layers that progressively reduce the dimensionality:

$$\text{Linear}(400 \rightarrow 300) \xrightarrow{\text{ReLU}} \text{Linear}(300 \rightarrow 200) \xrightarrow{\text{ReLU}} \text{Linear}(200 \rightarrow 100) \xrightarrow{\text{ReLU}} \text{Linear}(100 \rightarrow 1)$$

- **Final Output:** A single scalar representing the evaluation value Q for that specific frontier (e.g., how far the robot is from that frontier: 10 meters away $\rightarrow$ **Reward = -10**).



3.4 Detailed Decision-Making Process

The decision cycle is executed every time the robot reaches a target or requires a new destination. The procedure is rigorous:

1. **Frontier Detection:** The system scans the current map m_c to identify boundaries between free and unknown space. Filters are applied to remove frontiers that are too small (sensory noise) or unreachable. A candidate set $c = [p_1, p_2, \dots, p_N]$ is obtained.
2. **Global Processing:** The resized map m_r is generated and passed through the Global Network G to obtain the context vector $v_g = G(m_r)$. This computation is performed *only once* per decision step, as the global context is identical for all candidates.
3. **Iterative Candidate Evaluation:** For each frontier p_i in set c :
 - (a) The local map m_{Li} centered on p_i is cropped.
 - (b) The local feature vector $v_{Li} = L(m_{Li})$ is computed via the Local Network.
 - (c) The score $q_i = O(v_g, v_{Li})$ is computed via the Output Network, fusing the static global context with the specific local detail.
4. **Greedy Selection:** After evaluating all N candidates, the system selects the point p_g associated with the maximum q value: $p_g = p_k$ where $k = \text{argmax}(q)$ (e.g., $-10 > -39$), as seen in the previous section.

4 Reinforcement Learning Environment

4.1 Network Training via Double Deep Q-Learning (DDQN)

While the previous section described the LFPN architecture as the robot's "brain," this section explains how that brain is trained. The LFPN network acts as an approximator of the value function $Q(s, a; \theta)$, where θ represents the weights of the neural network. The goal of training is to optimize these weights so that the output Q value faithfully reflects the future negative distance, allowing the *greedy* algorithm described above to effectively select the best path.

To ensure learning stability in a complex environment like autonomous exploration, the **Double Deep Q-Learning (DDQN)** algorithm is adopted.

4.1.1 Dual Network Configuration

The DDQN method addresses the overestimation of Q-values typical of standard Q-Learning by decoupling action selection from its evaluation. To this end, two instances of the LFPN network are maintained, identical in structure but with distinct roles and parameters:

- **Policy Network (θ):** This is the main network used to make decisions during exploration and is continuously updated via gradient descent. It evolves very quickly and thus makes frequent errors (at each step it looks at the network's prediction, calculates how much it erred, and adapts). It is used for movement during training; during execution, it is frozen and used for deployment.
- **Target Network (θ'):** This is a more stable copy of the network, used to compute the target value during the loss calculation, preventing oscillations in learning. It evolves very slowly, updating once per episode by learning 3% of the **Policy Network** and correcting it at each step; during execution, it is discarded.

Initially, parameters θ and θ' are randomly initialized with the same values.

4.1.2 Loss Function and Optimization

During each training step, the agent samples a batch of experiences from the buffer (described below) and aims to minimize the error between the current network prediction and the calculated target. The peculiarity of DDQN lies in the target formula: the *Policy Network* decides which is the best next action a'_{max} in the next state s' , while the *Target Network* computes its value.

The next optimal action is selected as:

$$a'_{max} = \operatorname{argmax}_a Q(s', a; \theta) \quad (3)$$

The Loss Function $J(\theta)$ minimized via gradient descent is defined as:

$$J(\theta) = (R(s, a) + \gamma Q(s', a'_{max}; \theta') - Q(s, a; \theta))^2 \quad (4)$$

If the episode terminates in the next state, the future term is omitted and the target is simply the immediate reward $R(s, a)$.

4.1.3 Soft Target Update

Unlike traditional approaches that update the Target Network abruptly by copying weights every N steps, the authors adopt the *Soft Target Update* method, inspired by the DDPG algorithm. This ensures a much smoother and more stable update. At the end of each episode, the Target Network parameters are updated according to the formula:

$$\theta' \leftarrow \tau(\theta - \theta') + \theta' \quad (5)$$

The soft update factor τ is set to 0.03, meaning that the Target network changes very slowly, chasing the Policy Network with controlled inertia.

4.1.4 Data Management and Training Loop

Training efficiency is maximized through aggressive data management:

- **Experience (Replay Buffer):** The robot’s interactions are saved as tuples $\langle s, a, s', r, e \rangle$ in a buffer D^{Replay} , where e is a boolean indicating the end of the episode.
- **Training Frequency:** To accelerate convergence, the network is not trained only at the end of an episode; instead, 50 random datasets are sampled from the buffer **after every single exploration step**. This intensive approach explains the rapid convergence observed experimentally around episode 300.
- **Data Augmentation:** To improve the generalization capability of the LFPN network (so that it recognizes corridors and rooms regardless of orientation), the maps extracted from the buffer are rotated by 90° , 180° , or flipped horizontally/vertically before being used for training.

The entire process was executed on dedicated hardware (NVIDIA RTX 2080 Ti) for 600 total episodes (approximately 80 hours), using 4 maps of comparable sizes to keep the scale of rewards consistent.

5 Experiments and Results

5.1 Experimental Setup

The method was validated on 5 test maps with highly varied dimensions and aspect ratios (from $30 \times 14 \text{ m}^2$ to $28 \times 60 \text{ m}^2$). The following methods were compared:

1. **LFPN (Proposed)**
2. **DRL1 (Niroui et al.)**: Resizes the global map to 64×64 , uses weights for cell groups.
3. **DRL2 (Chen et al.)**: Uses only local maps of 20 m around the robot and 50 random targets.
4. **Nearest Frontier**: Chooses the closest frontier.
5. **Hybrid**: Combines travel cost and information gain with various weights.
6. **Nearest Frontier + RR**: Constantly rechecks the best target during movement.

5.2 Comparative Results

- **Distance Traveled**: The LFPN method completed the exploration with a shorter distance compared to all other methods across all test maps.
- **Large Environments**: The advantage of LFPN is most evident in vast environments (Map 5). While DRL1 and DRL2 explore quickly initially, they end up traveling greater total distances.
- **Generalization**: Despite training on small maps, LFPN showed excellent generalization capacity on large maps due to its architecture that preserves local details.

5.3 Physical Experiments

A real-world experiment was conducted on the 9th floor of the School of Automation (area $56 \times 55 \text{ m}^2$) using a robot equipped with Mecanum wheels and RPLIDAR.

- The robot completed the real exploration by traveling 445 m, a value very close to the 430 m obtained in the simulation of the same environment.
- This demonstrates that the method is easily transferable from the simulated environment to the real world (Sim-to-Real).

References

- [1] Rui Wang, Jie Zhang, Ming Lyu, Cheng Yan, Yaowei Chen (2024). *An improved frontier-based robot exploration strategy combined with deep reinforcement learning*. *Robotics and Autonomous Systems*, 181, 104783.
<https://doi.org/10.1016/j.robot.2024.104783>